CRAFTIFAI | FALLGUARD AI

Complete Build Plan
Product Requirements Document

Mandatory PipeGen track | Live webcam | NVIDIA GPU

DOCUMENT	VERSION	STATUS	DATE
Build PRD	1.0	Implementation-ready plan	17 Oct 2026

Project intent

Build a local, live-camera fall-risk demonstration that uses pose and motion over time to distinguish a possible fall from everyday bending or sitting. Use CraftifAI PipeGen v0.5.1 to document the full path from requirements through Pipeline IR and model selection to TensorRT compilation and live execution.

Hackathon success condition

Show every required PipeGen stage, compile at least one model to TensorRT, run on a live webcam, produce an actionable alert beyond boxes, and submit source code, PipeGen export, measured speed report, and Pipeline IR graph screenshots.

Prepared from the stated project brief and current official PipeGen and NVIDIA documentation. Hardware model, operating system, camera path, and final model are project decisions to verify during setup.

1. Executive Summary

FallGuard AI is a single-camera, single-person prototype for elder-care fall awareness. It runs locally on a PC or laptop with an NVIDIA GPU. Pose estimation supplies keypoints; temporal logic evaluates movement and posture; the user sees a clear state and reason. The prototype is an assistive demo and must not be represented as a clinically validated or emergency-response system.

Dimension	Decision for this build
Input	One live webcam; start with 640 x 480 or 720p at a stable frame rate.
Compute	NVIDIA x86 GPU; PipeGen v0.5.1; generated DeepStream runtime; TensorRT engine for at least one model.
Core model	Pose-estimation ONNX with documented keypoint format, compatible with the PipeGen path and its parser requirements.
Core behavior	Normal / Possible Fall / Confirmed Fall / Recovering / Unknown, with a human-readable trigger reason.
Privacy	Local processing; no continuous video recording; blur faces before any saved evidence frame.
Proof	Live demo, build/export artifacts, measured speed report, IR screenshot, test log, and source.

The key engineering risk is integration: PipeGen must support the selected pose model and its output decoding, and the live webcam must be visible to the generated runtime. Resolve both early with a minimal end-to-end spike before investing in UI polish.

2. Product Definition

Problem statement

An older adult may fall while alone, while ordinary activities such as bending to pick something up or sitting down can look similar in a single frame. A useful demo must interpret the sequence of pose changes over time and tell an observer what action is warranted.

Product outcome

A running webcam pipeline that identifies a sustained fall-like event, explains the evidence behind its state, and prompts a nearby caregiver or operator to check on the person. The demo prioritizes local operation, understandable behavior, and reproducible performance evidence.

Primary user

Hackathon presenter or evaluator viewing a live status panel and the pipeline evidence. Secondary stakeholder: caregiver/operator who needs a clear prompt, not raw model tensors.

Goals

- Complete and show all five PipeGen gates: Requirement, Pipeline IR, Model, Compiler, Run.
- Compile at least one model to a TensorRT engine and execute the generated pipeline live.
- Raise a distinct, actionable possible/confirmed fall status with an explanation and acknowledgement path.
- Demonstrate that ordinary bending and sitting do not immediately produce a confirmed-fall alert.
- Preserve a reproducible, privacy-conscious submission package with measured results.

Out of scope

- Clinical diagnosis, guaranteed fall detection, or claims of preventing injury.
- Automatic emergency-service calls, cloud video storage, multi-room deployment, identity recognition, continuous archival recording, or treating a box-only detector as fall detection.

3. Requirements and User Experience

Functional requirements

ID	Requirement	Priority	Acceptance evidence
FR-01	Acquire one live webcam stream and display a clear connected / disconnected state.	Must	Live image or pipeline preview; reconnect behavior documented.
FR-02	Run pose estimation and expose confidence-scored body keypoints for one person.	Must	Keypoint visualization or diagnostic output; TensorRT model build evidence.
FR-03	Track posture and movement over a temporal window; do not decide from a single frame.	Must	Event trace includes timestamps and state transitions.
FR-04	Report Normal, Possible Fall, Confirmed Fall, Recovering, and Unknown states.	Must	Visible status label and reason for every alert.
FR-05	Provide a local action: visible alert and acknowledge / reset control; optional sound.	Must	Operator can acknowledge and return to monitoring.
FR-06	Allow configuration of thresholds and confirmation / recovery durations.	Should	Values stored in readable config, with defaults and tuning notes.
FR-07	Keep an event log with timestamp, state, reason, and model/runtime metadata.	Should	Sample log contains no raw video or face identity data.
FR-08	If evidence images are saved, blur faces before persistence.	Must	Inspection confirms saved image is blurred; recording is off by default.

Status and action design

State	Interpretation	User-facing message / action
Normal	No fall-like pattern is present, or ordinary movement is stable.	Monitoring - no action required.
Possible Fall	A rapid descent or unexpected transition is detected; evidence is not yet persistent.	Please check the person. Continue evaluating; do not dispatch automatically.
Confirmed Fall	Fall-like transition plus sustained low / near-horizontal posture and no recovery through configured confirmation window.	Fall-like event detected. Check on the person and acknowledge.
Recovering	Person appears upright or resumes movement after an event.	Recovery detected. Review / acknowledge event, then resume monitoring.
Unknown	Camera lost, person not visible, keypoint confidence too low, or multiple people make tracking ambiguous.	Detection unavailable - check camera / scene.

Alert presentation should show the state, a concise reason (for example, 'rapid descent followed by sustained horizontal posture'), and elapsed confirmation time. The action should remain a human check-in. Do not imply certainty beyond the evidence or trigger an unattended emergency call.

4. Proposed Technical Design

Logical pipeline

Stage	Input / operation	Output
Camera	Live webcam capture; timestamp frames; report source health.	Frames + capture timestamps
Preprocess	Resize / normalize to the selected ONNX model's required input; maintain aspect ratio where required.	Model-ready tensor
Pose inference	Run a supported pose ONNX model compiled as a TensorRT engine.	Keypoints, confidence, optional person region
Track + temporal logic	Associate the person across frames; calculate body center, torso angle, vertical displacement, posture duration, and confidence.	State transitions + evidence features
Alert + output	Render status and reason; optional local tone; acknowledge / reset; emit minimal event log.	Actionable local user output

The exact PipeGen graph, DeepStream element mapping, and custom parsing needs depend on the selected model and v0.5.1 build. Treat the graph above as the product requirement; use the generated Pipeline IR as the implementation contract and capture its actual graph for submission.

Pose and fall-state logic

- Use pose landmarks such as shoulders, hips, knees, and ankles. Normalize position and angles by person scale so thresholds are less dependent on image size.
- Candidate features: sudden downward body-center displacement, torso angle approaching horizontal, low hip/shoulder position, and duration of the resulting posture.
- Possible Fall can begin on a rapid unexpected descent. Confirm only when the low / near-horizontal posture persists and the person does not recover within a tuned window.
- Bending is more likely a gradual change with the torso moving down while hips / knees and later body posture indicate recovery; sitting typically settles into a stable seated pose. These are heuristics to validate, not guarantees.
- Use confidence gates, hysteresis, recovery transitions, and alert cooldown. If keypoints or tracking are unreliable, show Unknown rather than silently declaring Normal.
- Tune thresholds using controlled clips and the actual camera viewpoint. Do not encode an unvalidated duration or angle as a universal fall threshold.

5. PipeGen Track Execution Plan

Complete these five stages in order. Save proof from the actual PipeGen session and export, not recreated screenshots or estimated performance figures.

Stage	Build activity	Exit gate / retained evidence
1. Requirement	Enter the project goal, source (live webcam), target PC/NVIDIA GPU, pose inference, temporal logic, states/actions, privacy limits, and performance intent. Specify one stream and a one-person initial demo.	Requirement text saved or screenshot. Confirm accepted pipeline/task mapping or decide BYO repository/model.
2. Pipeline IR	Inspect the generated hardware-independent graph. Check camera, preprocess, model, postprocess/keypoint decode, temporal logic, output and runtime connections. Resolve missing nodes before compile.	Screenshot of the complete node graph; exported IR/config if available; notes on any generated graph gaps.
3. Model	Select a PipeGen-supported pose pipeline or provide a compatible repository and ONNX. Inspect input dimensions, output tensors, keypoint order, preprocessing, and parser support. Avoid a box-only model as the sole fall signal.	Model capability/profile evidence; source/model license and version; successful compatibility check.
4. Compiler	Compile and validate the full pipeline. Ensure at least one model becomes a TensorRT engine. Fix model/parser/config errors using PipeGen's build feedback and retain the successful export.	Successful compile/build record; TensorRT engine in export; compiler logs; PipeGen speed report.
5. Run	Run generated `run.sh`/launch instructions with the live webcam. Confirm GPU inference, status updates, event behavior, camera-loss state, and local alert action.	Live run recording or demo; measured performance; event test log; final source + PipeGen export.

Build gates

Gate	Pass condition	If it fails
G0 - Platform	PipeGen launches; GPU visible; Docker/runtime ready; adequate disk; webcam is accessible from runtime.	Fix platform/camera path first. Do not spend time tuning fall logic until frames reach the pipeline.
G1 - Model	Pose model loads; keypoints can be decoded with correct input/output semantics.	Try the PipeGen curated match or a supported ONNX; verify parser and preprocessing assumptions.
G2 - Live	Continuous frames reach inference and the UI reports camera health.	Use low resolution/single stream; isolate capture from inference; inspect generated source and runtime logs.
G3 - Behavior	Normal, bending, sitting, lie-down, simulated fall, recovery, and occlusion produce expected states.	Tune on recorded test sequences and inspect features/transitions; do not add untested aggressive triggers.
G4 - Submission	All stage evidence and required files exist in one organized package; report numbers match measured run.	Use the final submission checklist in Section 10.

6. Platform Setup

Preferred path for a Windows 11 NVIDIA laptop: use the PipeGen v0.5.1 WSL2 instructions and Ubuntu 24.04. PipeGen's published page lists a 560.76-or-newer Windows host driver for WSL2 and requires an X11 graphical session. NVIDIA's current DeepStream WSL guide also specifies Windows 11, Ubuntu 24.04, and a Windows NVIDIA driver. Since PipeGen downloads and runs its own DeepStream container, follow PipeGen's versioned install path instead of independently mixing DeepStream/CUDA/TensorRT packages.

Step	Action	Check
1. Check host	Update/install NVIDIA Windows driver for the GPU; keep at least 40-50 GB free for download, unpacked container, and project files.	PowerShell: `nvidia-smi`; confirm driver meets PipeGen's WSL2 minimum.
2. Enable WSL2	PowerShell as Administrator: `wsl --install -d Ubuntu-24.04` then `wsl --update`. Restart if asked; start Ubuntu and create its user.	`wsl --list --verbose` shows Ubuntu version 2.

Step	Action	Check
3. Check GPU + display	In Ubuntu: `nvidia-smi`; inspect available disk with `df -h`; verify PipeGen's required X11 GUI session per official WSL tab.	GPU is visible in Linux; enough disk; graphical PipeGen app can be launched.
4. Install PipeGen	Download the v0.5.1 WSL2 package and run the WSL-specific dependency/setup instructions from the official PipeGen page. Install its downloaded `.deb` using the package filename provided.	PipeGen installed; Docker available; user group changes applied by relog/restarting distro if requested.
5. First launch	Run `pipegen` from a graphical Linux terminal. Allow first-run DeepStream image download and unpacking; retry launch if download is interrupted.	PipeGen sign-in succeeds; project opens; generated runtime is ready.
6. Verify camera	Check that the selected webcam can be opened from the Linux/DeepStream execution context before selecting it in the pipeline.	A live frame reaches the pipeline. If not, use PipeGen's WSL-specific camera guidance or native Ubuntu.

For native Ubuntu x86_64, the official PipeGen page gives a prerequisite script followed by `sudo apt install ./PipeGen_0.5.1_amd64.deb`; run the downloaded script and package from the official Linux x86_64 tab. Its page lists a 560.35.03-or-newer Linux driver and 50 GB free space. Log out and back in after setup if it adds your user to the Docker group.

Preflight commands (Ubuntu / WSL Ubuntu)

```
nvidia-smi
df -h
python3 --version
echo $DISPLAY
docker info
```

The `DISPLAY` and `docker info` checks are diagnostics, not a replacement for the PipeGen installer. Do not copy a generic DeepStream installation recipe into this environment; use the PipeGen-provided setup and its pinned runtime image.

7. Implementation Work Packages

WP	Work	Deliverable
WP0 - Setup spike	Install PipeGen, verify GPU/display/container, and prove webcam input. Record OS, GPU, driver, PipeGen version, free disk.	Preflight notes; screenshot of PipeGen running; camera proof.
WP1 - Requirement + IR	Enter detailed requirement; inspect graph; capture IR; note what PipeGen generated versus what needs model/parser/custom temporal logic support.	Requirement and IR evidence.
WP2 - Pose inference	Select supported pose ONNX and verify tensors, keypoints, preprocessing, parser; compile to TensorRT.	Engine + model metadata + successful compilation evidence.
WP3 - Event logic	Build/tune state machine from keypoints and time windows; add reason text, acknowledge/recovery/cooldown.	State logic/config; event traces across scenarios.
WP4 - Runtime UX	Run live webcam, show status and camera health, add local alert/acknowledge. Keep capture local.	Working live run and action output.
WP5 - Validation + package	Test scenario matrix; measure FPS/latency; capture report and graph; package source and PipeGen export.	Submission folder ready for review.

8. Validation and Performance

Test with a consenting volunteer in a clear, controlled space. Simulated falls should be performed safely, with a spotter and soft surface. Keep faces blurred in any saved frame and avoid storing video unless a test specifically requires it.

Scenario	Expected result	Record
Standing / walking	Normal; no fall alert.	False alert count; FPS; latency.
Slow bend and return	No Confirmed Fall; possible transient Unknown only if pose confidence drops.	State sequence; trigger reason if any.
Sit and remain seated	No Confirmed Fall after settling into a stable seated posture.	State sequence; false positive.
Lie down deliberately	May become Possible Fall; should follow configured confirmation behavior and be easy to acknowledge.	Detection delay; state transitions.
Controlled simulated fall	Possible Fall then Confirmed Fall if pose evidence persists; display explicit check-in action.	Detection delay; misses/false trigger; reason.
Recovery after confirmed state	Recovering then Normal after operator acknowledgement / configured reset.	Recovery delay; cooldown.
Person partly occluded / low light	Unknown or degraded-confidence status; avoid claiming Normal when pose is unavailable.	Confidence; status and runtime behavior.
Camera disconnected	Camera unavailable state; alert stops relying on stale frames; reconnect behavior is clear.	Time to detect loss/recovery.
No person / multiple people	No person: monitoring or Unknown. Multiple people: maintain declared one-person scope or show ambiguity.	Tracking behavior; ambiguity handling.

Performance measurement protocol

- Use the PipeGen-generated speed report from the final compiled run. Save the original report and include a copy in the submission evidence folder.
- Record GPU model, driver, OS/WSL, PipeGen version, model/precision, input resolution, camera FPS, measured pipeline FPS, and end-to-end latency if reported.
- Warm up the pipeline before measurement, then measure a fixed interval with the same camera/resolution used in the demo. Record the interval and any dropped frames.
- Report measured values only. Keep model inference time distinct from end-to-end camera-to-alert latency when both are available.
- Compare accuracy/behavior to the controlled scenario log. A high FPS result does not prove fall-state quality.

Acceptance targets

Area	Target
Pipeline	One live webcam stream runs continuously through the compiled model on the NVIDIA GPU.
PipeGen compliance	All five named stages completed and evidenced; at least one TensorRT engine present in the export.
User output	Status, reason, and human action prompt visible; acknowledgement/recovery behavior demonstrated.
Scenario quality	Bending and ordinary sitting do not become Confirmed Fall in the demonstration matrix; simulated fall can reach Confirmed Fall when configured evidence persists.
Performance	PipeGen speed report saved with hardware/runtime context; project team states measured FPS/latency without inventing a score target.
Privacy	No continuous raw-video storage. Any saved evidence frame has face blur applied before file write.

9. Privacy, Safety, and Failure Handling

Privacy requirements

- Process frames locally on the NVIDIA device. Do not stream video to a cloud service for the core demo.
- Default to no video recording. Keep only minimal event metadata needed for debugging or judging (timestamp, state, reason, confidence summary, model/runtime version).
- If a frame must be saved, run face detection and blur before encoding/writing; verify the saved artifact itself. Do not save the unblurred original in a temp folder or debug cache.
- Use consented volunteers and remove test captures after judging unless retention is explicitly required.
- Do not collect identity, audio, or unrelated personal data. Avoid showing identifiable footage in submission screenshots.

Safety and reliability

- Display 'Fall-like event' or 'Possible/Confirmed Fall' rather than presenting a medical diagnosis.
- Every alert asks a human to check on the person. No automatic emergency contact in this prototype.
- Treat camera loss, low confidence, occlusion, and ambiguous multi-person tracking as degraded/Unknown states.
- Do not suppress a new event solely because a previous alert was acknowledged; use a defined cooldown and re-arm condition.
- Show when the pipeline is starting, disconnected, or unable to infer; do not show a stale 'Normal' state as if current.

10. Risks and Mitigations

Risk	Impact	Mitigation / decision point
Pose pipeline not in curated library	Model stage stalls or output decode is wrong.	Check curated matches first; use a known compatible ONNX/repository and confirm parser support before building UX.
Webcam not visible in WSL runtime	No live Run-stage demo.	Test camera in WP0; follow official PipeGen WSL camera guidance; use native Ubuntu if necessary.
PipeGen UI requires X11	App fails to launch in Linux/WSL GUI.	Check official X11/session requirements and WSL setup; validate app launch before downloading large models.
TensorRT conversion incompatibility	Compiler fails or output semantics change.	Follow PipeGen's generated compile path; check ONNX ops, dynamic shapes, precision and calibration needs; compare outputs after conversion.
Bend/sit false positives	Loss of trust; demo failure.	Use temporal transitions, relative geometry, confidence, and recovery logic; test ordinary movements before final demo.
Fall missed due to view/occlusion	False reassurance.	State camera limitations; provide Unknown on weak evidence; test camera placement and frame coverage.
Insufficient storage/network	First run cannot fetch or unpack image.	Reserve 40-50 GB; use stable internet; allow image download; keep export/evidence space.
Privacy leak in demo capture	Sensitive face/video included in output.	Disable recording by default; blur before writing; inspect evidence package before submission.
Performance report absent	Mandatory PipeGen evidence incomplete.	Confirm report location in generated export/UI immediately after Run; retain the original and record context.

11. Schedule and Deliverables

Suggested sequence for a short hackathon build; adjust durations to the event window. Run the setup and camera spike first because it can invalidate the chosen platform path.

Milestone	Suggested effort	Completion evidence
M0 - Preflight	1-2 hours	PipeGen starts; GPU/runtime visible; camera can deliver frames; platform versions recorded.
M1 - Requirement + IR	1-2 hours	Requirement accepted; IR graph reviewed and screenshot saved.
M2 - Model + compile	2-4 hours	Pose model validated; TensorRT build completed; output/exports retained.
M3 - Temporal logic	3-6 hours	State machine and explanation output implemented; core motion cases pass initial tests.
M4 - Live run + tuning	2-4 hours	Live webcam demo; test matrix logged; thresholds tuned for demo camera.
M5 - Submission pack	1-2 hours	Speed report, IR screenshot, source, PipeGen export, README and privacy review ready.

Repository / submission layout

fallguard-ai/
README.md
src/ # state logic, UI glue, config
models/ # model metadata; large weights only if permitted
config/ # camera, model and temporal thresholds
tests/ # scenario notes or test scripts
pipegen-export/ # complete generated export, including run script and engine
evidence/
pipegen-requirement.png
pipegen-pipeline-ir.png
pipegen-model-tensorrt.png
pipegen-run.png
pipegen-speed-report.*
scenario-test-log.csv
privacy/README.md # what is recorded and retention policy

README minimum contents

- Project purpose, tested hardware/OS/driver/PipeGen version, and exact launch steps.
- Model name/version, license/source, ONNX input/output notes, keypoint mapping, and TensorRT build evidence.
- Camera selection and resolution, run command from the PipeGen export, and known limitations.
- Measured speed report values with measurement conditions; link to the original report file.
- Fall-state logic and configured thresholds; scenario matrix and outcome summary.
- Privacy policy: no continuous video retention; face blur applied before saving any frame.

12. Demo Script and Submission Checklist

Demo sequence

- Show PipeGen Requirement and state the target: live webcam, NVIDIA GPU, pose + temporal fall logic.
- Show the Pipeline IR graph and identify camera, model, temporal decision, and actionable output nodes.
- Show the selected pose model and successful TensorRT compilation evidence.
- Run the live webcam. Demonstrate normal walking, a slow bend, sitting, then a safe simulated fall and recovery.
- Explain the alert state and reason; acknowledge it as the human operator. Demonstrate camera loss or Unknown if time allows.
- Open the measured speed report and state its hardware/resolution context. Finish by showing source and export folder contents.

Check	Done
All five PipeGen stages shown: Requirement, Pipeline IR, Model, Compiler, Run.	[ ]
At least one model compiled to TensorRT; compiled engine included in export.	[ ]
Live webcam inference runs on NVIDIA GPU; camera availability checked from runtime.	[ ]
Actionable status and reason displayed beyond bounding boxes; acknowledgement shown.	[ ]
Bending and sitting tested; safe simulated fall and recovery tested; outcomes logged.	[ ]
PipeGen speed report retained; reported numbers are measured and contextualized.	[ ]
Pipeline IR node-graph screenshot retained; clear and legible.	[ ]
Complete PipeGen export and source code included; run instructions work.	[ ]
No continuous raw video; any saved evidence frame is blurred before saving.	[ ]
README states limitations and that this is not a validated medical alarm.	[ ]

13. Assumptions, Open Decisions, and References

Assumptions

- Target hardware is an x86 PC/laptop with an NVIDIA GPU; exact GPU model and OS are not yet specified.
- The user can install PipeGen v0.5.1 and create/sign in to a CraftifAI account.
- The hackathon requires all five stages, at least one TensorRT-compiled model, live run, a PipeGen speed report, Pipeline IR screenshots, source code, and PipeGen export; this PRD treats those as mandatory acceptance criteria.
- A pose ONNX model and decoding path can be selected from PipeGen's curated library or supplied by the team. Exact model support must be confirmed in the app.
- The app can produce a status output in the generated pipeline or through a small local companion UI, subject to what the PipeGen export supports.

Open decisions to resolve during WP0 / WP1

Decision	Default for this PRD
OS: native Ubuntu or Windows WSL2?	Windows 11 WSL2 if X11 and camera path work; native Ubuntu fallback.
Exact GPU, driver, memory?	Capture from `nvidia-smi`; check support before model selection.
Webcam type and connection?	One built-in or USB webcam, verified inside runtime before model work.
Pose model and parser?	PipeGen-supported pose pipeline preferred; otherwise compatible ONNX with documented outputs.
What is the exact location/format of speed report in v0.5.1?	Locate in actual build UI/export; do not substitute hand-calculated performance.
Does PipeGen provide temporal logic/output node directly?	Inspect Pipeline IR; add supported custom code/repository only if required and permitted by track.

Official references

CraftifAI, PipeGen v0.5.1 product, requirements, setup and workflow. Used for driver/disk/display requirements, first-launch container size, installation flow, IR, supported pipeline choice, and generated export workflow.

NVIDIA, DeepStream on WSL. Used for Windows 11 / WSL2 / Ubuntu 24.04 platform context and host-driver behavior.

NVIDIA, DeepStream Installation Guide. Current DeepStream releases have specific OS, CUDA, TensorRT, and driver matrices; the PRD therefore directs the team to PipeGen’s versioned bundled runtime rather than mixing independent versions.

NVIDIA, TensorRT Prerequisites. Used for current TensorRT/CUDA/driver compatibility guidance.

End of document